LATIHAN TERSTRUKTUR

Pertemuan 4

Implementasi Unit Testing dengan GitHub Actions

Nama	:	Jacky Firmansah
NIM	:	1224010
Tanggal	:	12 Oktober 2026

Bagian A: Git & GitHub Dasar

Soal 1

1. Centralized VCS vs Distributed VCS

	Centralized VCS	Distributed VCS
Penyimpanan riwayat	Seluruh riwayat ada di satu server pusat. Developer hanya mengambil versi yang sedang dikerjakan.	Setiap developer punya salinan penuh repository beserta seluruh riwayatnya.
Kerja offline	Hampir tidak bisa. Commit butuh koneksi ke server.	Bisa. Commit, lihat log, dan buat branch dilakukan di lokal, baru disinkronkan nanti.
Kalau server mati	Kerja tim terhenti, dan data bisa hilang kalau tidak ada backup.	Salinan di komputer mana pun bisa dipakai memulihkan repository.
Contoh	Subversion (SVN), CVS	Git, Mercurial

2. Lima Git command yang wajib dikuasai

Command	Fungsi
`git init`	Membuat repository Git baru di folder yang sedang dibuka.
`git add`	Memasukkan perubahan file ke staging area sebelum di-commit.
`git commit -m "pesan"`	Menyimpan isi staging area sebagai satu snapshot di riwayat repository.
`git push`	Mengirim commit lokal ke repository remote, misalnya GitHub.
`git pull`	Mengambil perubahan terbaru dari remote lalu menggabungkannya ke branch lokal.

Command lain yang sering dipakai: `git clone`, `git status`, `git branch`, `git checkout` dan `git merge`.

3. Fungsi .gitignore

File `.gitignore` berisi daftar pola nama file atau folder yang tidak boleh dilacak Git. Isinya biasanya hasil build seperti `target/` dan `*.class`, folder IDE seperti `.idea/`, serta file rahasia seperti `.env`. Dengan begitu repository tetap bersih, ukurannya tidak membengkak, dan password atau API key tidak ikut terunggah ke GitHub.

Soal 2: Praktik Git Commands

Berikut command yang digunakan secara berurutan sesuai langkah praktikum:

No	Langkah	Command
1	Buat repository di web GitHub: latihan-git-1224010	Melalui web GitHub: Buat repository baru dengan nama `latihan-git-1224010`
2	Clone repository ke lokal	`git clone https://github.com/jackalwyssad/latihan-git-1224010.git` `cd latihan-git-1224010`
3	Buat file README.md (Nama: Jacky Firmansah, NIM: 1224010)	`echo "Nama: Jacky Firmansah, NIM: 1224010" > README.md`

No	Langkah	Command
4	Commit README	`git add README.md` `git commit -m "docs: add README"`
5	Push ke GitHub	`git branch -M main` `git push -u origin main`
6 & 7	Buat branch feature-test dan pindah ke branch tersebut	`git checkout -b feature-test`

8	Buat file test.txt dan commit	`echo "file untuk percobaan" > test.txt`
		`git add test.txt`
		`git commit -m "test: add test file"`

9	Push branch feature-test ke GitHub	`git push -u origin feature-test`

10	Merge branch feature-test ke main	`git checkout main`
		`git merge feature-test`
		`git push origin main`

Untuk memastikan hasilnya benar, verifikasi dilakukan dengan `git log --oneline`. Commit dari branch feature-test sudah muncul dan berhasil digabungkan ke branch main.

Bagian B: Unit Testing Framework

Soal 3: Konversi Test Case ke Kode Unit Test

Method `validasiPendaftaran` punya lima titik keputusan (P1 sampai P5). Saya anggap method ini berada di class `Pendaftaran`. Lima independent path-nya disusun berdasarkan titik keputusan itu, dari yang paling awal sampai jalur sukses.

1. Test case berdasarkan independent path

Path	Jalur keputusan	TC	Nama	Umur	Email	Setuju	Hasil yang diharapkan
1	P1 benar	TC1	"" (kosong)	20	budi@mail.com	true	Nama harus diisi
2	P1 salah, P2 benar	TC2	Budi	16	budi@mail.com	true	Umur minimal 17 tahun
3	P1, P2 salah, P3/P4 benar	TC3	Budi	20	budi.mail.com	true	Email tidak valid
4	P1 sampai P4 salah, P5 benar	TC4	Budi	20	budi@mail.com	false	Harus menyetujui syarat dan ketentuan
5	Semua kondisi salah	TC5	Budi	20	budi@mail.com	true	Pendaftaran berhasil

P1 dan P3/P4 berisi kondisi gabungan dengan operator ||. Karena itu saya tambah tiga test pelengkap di kode: nama `null`, email `null`, dan umur tepat 17 sebagai nilai batas.

2. Implementasi JUnit

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class PendaftaranTest {

    private Pendaftaran pendaftaran;

    @BeforeEach
    void setUp() {
        pendaftaran = new Pendaftaran();
    }

    @Test
    @DisplayName("TC1: nama kosong")
    void namaKosong() {
        assertEquals("Nama harus diisi",
                pendaftaran.validasiPendaftaran("", 20, "budi@mail.com", true));
    }

    @Test
    @DisplayName("TC2: umur di bawah 17")
    void umurKurangDari17() {
        assertEquals("Umur minimal 17 tahun",
                pendaftaran.validasiPendaftaran("Budi", 16, "budi@mail.com", true));
    }

    @Test
    @DisplayName("TC3: email tanpa @")
    void emailTidakValid() {
        assertEquals("Email tidak valid",
                pendaftaran.validasiPendaftaran("Budi", 20, "budi.mail.com", true));
```

```
    }

    @Test
    @DisplayName("TC4: syarat tidak disetujui")
    void syaratTidakDisetujui() {
        assertEquals("Harus menyetujui syarat dan ketentuan",
                pendaftaran.validasiPendaftaran("Budi", 20, "budi@mail.com", false));
    }

    @Test
    @DisplayName("TC5: semua data valid")
    void pendaftaranBerhasil() {
        assertEquals("Pendaftaran  berhasil",
                pendaftaran.validasiPendaftaran("Budi", 20, "budi@mail.com", true));
    }

    // ---- test pelengkap untuk kondisi gabungan dan nilai batas ----

    @Test
    @DisplayName("Tambahan: nama null")
    void namaNull() {
        assertEquals("Nama harus diisi",
                pendaftaran.validasiPendaftaran(null, 20, "budi@mail.com", true));
    }

    @Test
    @DisplayName("Tambahan: email null")
    void emailNull() {
        assertEquals("Email tidak valid",
                pendaftaran.validasiPendaftaran("Budi", 20, null, true));
    }

    @Test
    @DisplayName("Tambahan: umur tepat 17 diterima")
    void umurBatas17() {
        assertEquals("Pendaftaran  berhasil",
                pendaftaran.validasiPendaftaran("Budi", 17, "budi@mail.com", true));
    }
}
```

Soal 4: Unit Test untuk StudentGrade

1. Cyclomatic Complexity calculateGrade()

Titik keputusan di method ini ada enam: dua kondisi di validasi (`score < 0` dan `score > 100`) dan empat percabangan nilai (`>= 80`, `>= 70`, `>= 60`, `>= 50`).

CC = jumlah predicate + 1 = 6 + 1 = 7

Kalau operator `||` di validasi dihitung sebagai satu kondisi saja, hasilnya 6. Saya pakai 7 karena tiap kondisi dihitung terpisah, dan angka ini cocok dengan jumlah jalur nyata di bawah.

2. Independent path

Path	Jalur	Contoh input	Hasil
1	score < 0	-1	IllegalArgumentException
2	score >= 0, score > 100	101	IllegalArgumentException
3	valid, score >= 80	85	A
4	valid, < 80, >= 70	75	B
5	valid, < 70, >= 60	65	C
6	valid, < 60, >= 50	55	D
7	valid, < 50	30	E

3. Implementasi JUnit untuk calculateGrade dan isPassed

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import static org.junit.jupiter.api.Assertions.*;

class StudentGradeTest {

    private StudentGrade sg;

    @BeforeEach
    void setUp() {
        sg = new StudentGrade();
    }

    // ---------- calculateGrade: satu test per independent path ----------

    @Test
    void calculateGrade_scoreNegatif_melemparException() {
        assertThrows(IllegalArgumentException.class, () -> sg.calculateGrade(-1));
    }

    @Test
    void calculateGrade_scoreDiAtas100_melemparException() {
        assertThrows(IllegalArgumentException.class, () -> sg.calculateGrade(101));
    }

    @Test
    void calculateGrade_85_adalahA() {
        assertEquals("A", sg.calculateGrade(85));
    }

    @Test
    void calculateGrade_75_adalahB() {
        assertEquals("B", sg.calculateGrade(75));
    }

    @Test
    void calculateGrade_65_adalahC() {
        assertEquals("C", sg.calculateGrade(65));
    }

    @Test
    void calculateGrade_55_adalahD() {
        assertEquals("D", sg.calculateGrade(55));
    }

    @Test
    void calculateGrade_30_adalahE() {
        assertEquals("E", sg.calculateGrade(30));
    }

    // ---------- calculateGrade: nilai batas ----------

    @ParameterizedTest
    @CsvSource({
        "0,E", "49,E", "50,D", "59,D", "60,C", "69,C",
        "70,B", "79,B", "80,A", "100,A"
    })
    void calculateGrade_nilaiBatas(int score, String expected) {
        assertEquals(expected, sg.calculateGrade(score));
    }

    // -------------- isPassed --------------

    @Test
    void isPassed_60_lulus() {
```

```
        assertTrue(sg.isPassed(60));
    }

    @Test
    void isPassed_59_tidakLulus() {
        assertFalse(sg.isPassed(59));
    }

    @Test
    void isPassed_100_lulus() {
        assertTrue(sg.isPassed(100));
    }

    @Test
    void isPassed_0_tidakLulus() {
        assertFalse(sg.isPassed(0));
    }

    @Test
    void isPassed_scoreNegatif_melemparException() {
        assertThrows(IllegalArgumentException.class, () -> sg.isPassed(-1));
    }

    @Test
    void isPassed_scoreDiAtas100_melemparException() {
        assertThrows(IllegalArgumentException.class, () -> sg.isPassed(101));
    }
}
```

Test `@ParameterizedTest` butuh dependency `junit-jupiter` (sudah termasuk di pom.xml pada Bagian E).

Bagian C: GitHub Actions

Soal 5: Analisis Workflow YAML

1. Nama workflow

Nama workflow-nya **Python CI**, diambil dari baris `name: Python CI`.

2. Trigger yang memicu workflow

- Event `push` ke branch `main` atau `develop`.
- Event `pull_request` yang menuju branch `main`.

3. Fungsi strategy matrix

Matrix membuat job `test` dijalankan berulang kali dengan nilai variabel yang berbeda. Di sini variabelnya `python-version` dengan isi 3.10, 3.11, dan 3.12, jadi GitHub Actions menjalankan tiga job yang paralel, masing-masing di versi Python berbeda. Nilainya dipakai lewat `${{ matrix.python-version }}`.

Manfaatnya, kita bisa tahu apakah kode lolos test di ketiga versi tanpa menulis tiga job terpisah. Kalau satu versi gagal, kelihatan jelas versi mana yang bermasalah.

4. Step di dalam job test

No	Nama step	Yang dilakukan
1	Checkout code	Mengambil isi repository ke runner (`actions/checkout@v4`).
2	Setup Python ${{ matrix.python-version }}	Memasang versi Python sesuai matrix (`actions/setup-python@v5`).
3	Install dependencies	Upgrade pip, lalu memasang pytest dan pytest-cov.
4	Run tests	Menjalankan `pytest --cov=src --cov-report=xml` dan menghasilkan laporan coverage berformat XML.

Soal 6: Workflow YAML untuk Project Java

File disimpan di `.github/workflows/java-test.yml`.

```
name: Java Unit Test

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
```

```
        java-version: '17'
        cache: maven

    - name: Run tests
      run: mvn test

    - name: Generate coverage report
      run: mvn jacoco:report

    - name: Upload JaCoCo report
      uses: actions/upload-artifact@v4
      with:
        name: jacoco-report
        path: target/site/jacoco/
```

Perintah `mvn jacoco:report` hanya menghasilkan laporan kalau plugin JaCoCo sudah dipasang di `pom.xml` dengan goal `prepare-agent`, supaya data eksekusi test terekam..

Bagian D: Code Coverage & Integrasi

Soal 7: Analisis Code Coverage

1. Total statement

10 + 15 + 20 + 5 = **50 statement**.

2. Total statement yang tidak tereksekusi (Miss)

0 + 2 + 8 + 1 = **11 statement**. Cek silang: (50 - 11) / 50 = 78%, sama dengan angka TOTAL di laporan.

3. File dengan coverage terendah

src/payment.py dengan coverage **60%** (8 dari 20 statement tidak tereksekusi).

4. Cara meningkatkan coverage payment.py

- Jalankan `pytest --cov=src --cov-report=term-missing` supaya kolom Missing menampilkan nomor baris dari 8 statement yang belum tersentuh.
- Lihat baris-baris itu. Biasanya isinya cabang yang jarang jalan: error handling, saldo tidak cukup, metode pembayaran tidak valid, atau blok `except`.
- Tulis test baru untuk tiap cabang tadi, termasuk input tidak valid dan nilai batas. Kalau ada dependensi eksternal seperti gateway pembayaran, ganti dengan mock.
- Jalankan ulang dan pastikan angkanya naik. Targetnya minimal 80%.
- Kalau ada baris yang ternyata tidak mungkin dicapai (dead code), hapus kodenya, jangan dipaksakan lewat test.

5. Apakah 78% sudah cukup baik?

Menurut saya belum, walaupun sudah dekat. Banyak tim memakai 80% sebagai batas minimal, jadi 78% masih sedikit di bawahnya. Yang lebih mengganjal adalah sebarannya. calculator.py sudah 100%, tetapi payment.py, modul yang mengurus uang, justru paling rendah di 60%. Bagian paling berisiko malah paling sedikit diuji.

Satu hal lagi: coverage hanya mengukur baris yang dijalankan, bukan apakah assertion-nya benar. Angka tinggi tidak otomatis berarti kode aman. Jadi prioritas saya adalah menaikkan payment.py dulu, baru mengejar angka total.

Bagian E: Studi Kasus Terpadu (Bonus)

Soal 8: Project BankAccount

Struktur project

```
latihan-bankaccount/
  .github/workflows/java-test.yml
  src/main/java/BankAccount.java
  src/test/java/BankAccountTest.java
  pom.xml
  .gitignore
```

1. Kode program: BankAccount.java

```
public class BankAccount {

    private double balance;

    public BankAccount() {
        this(0);
    }

    public BankAccount(double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Saldo awal tidak boleh negatif");
        }
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Jumlah deposit tidak boleh negatif");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Jumlah penarikan tidak boleh negatif");
        }
        if (amount > balance) {
            throw new IllegalArgumentException("Saldo tidak mencukupi");
        }
        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

2. Unit test: BankAccountTest.java (10 test case)

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class BankAccountTest {

    private static final double DELTA = 0.001;
    private BankAccount account;

    @BeforeEach
    void setUp() {
        account = new BankAccount(100);
    }

    @Test
```

```
    void rekeningBaruTanpaSaldoAwal_saldoNol() {
        assertEquals(0, new BankAccount().getBalance(), DELTA);
    }

    @Test
    void saldoAwalNegatif_melemparException() {
        assertThrows(IllegalArgumentException.class, () -> new BankAccount(-1));
    }

    @Test
    void deposit_menambahSaldo() {
        account.deposit(50);
        assertEquals(150, account.getBalance(), DELTA);
    }

    @Test
    void deposit_nol_saldoTetap() {
        account.deposit(0);
        assertEquals(100, account.getBalance(), DELTA);
    }

    @Test
    void deposit_negatif_melemparException() {
        assertThrows(IllegalArgumentException.class, () -> account.deposit(-10));
    }

    @Test
    void withdraw_mengurangiSaldo() {
        account.withdraw(40);
        assertEquals(60, account.getBalance(), DELTA);
    }

    @Test
    void withdraw_sebesarSaldo_saldoMenjadiNol() {
        account.withdraw(100);
        assertEquals(0, account.getBalance(), DELTA);
    }

    @Test
    void withdraw_negatif_melemparException() {
        assertThrows(IllegalArgumentException.class, () -> account.withdraw(-5));
    }

    @Test
    void withdraw_melebihiSaldo_melemparExceptionDanSaldoTidakBerubah() {
        assertThrows(IllegalArgumentException.class, () -> account.withdraw(100.01));
        assertEquals(100, account.getBalance(), DELTA);
    }

    @Test
    void beberapaTransaksiBerurutan_saldoAkhirBenar() {
        account.deposit(200);
        account.withdraw(50);
        account.deposit(25);
        assertEquals(275, account.getBalance(), DELTA);
    }
}
```

3. GitHub Actions

Workflow-nya sama dengan yang ditulis di Soal 6 (`.github/workflows/java-test.yml`): test dijalankan dengan `mvn test`, laporan dibuat lewat JaCoCo, lalu diunggah sebagai artifact `jacoco-report`. Supaya JaCoCo bekerja, `pom.xml` berisi konfigurasi berikut.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
```

```
  <groupId>com.example</groupId>
  <artifactId>latihan-bankaccount</artifactId>
  <version>1.0</version>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

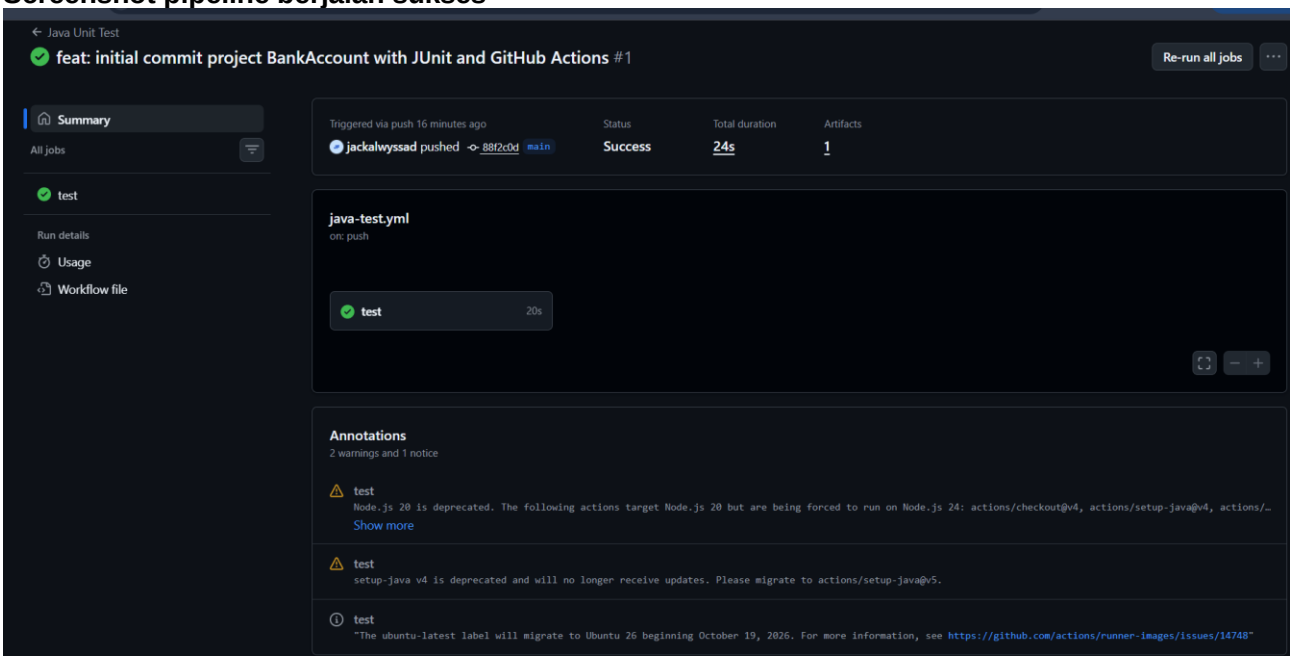
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
      </plugin>
      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.12</version>
        <executions>
          <execution>
            <goals><goal>prepare-agent</goal></goals>
          </execution>
          <execution>
            <id>report</id>
            <phase>test</phase>
            <goals><goal>report</goal></goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

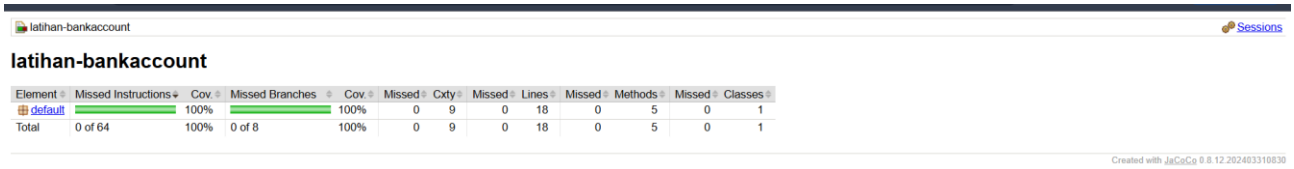
4. Repository dan bukti

Link repository, screenshot pipeline, dan screenshot coverage diisi setelah project diunggah ke GitHub dan pipeline-nya berjalan sukses.

Screenshot pipeline berjalan sukses



Screenshot coverage report



Link Repository GitHub

Repository latihan Git (Soal 2):

https://github.com/jackalwyssad/latihan-git-1224010

Repository studi kasus BankAccount (Soal 8, bonus):

https://github.com/jackalwyssad/latihan-bankaccount